

Bank Account Fraud Detection: Fraud Detection with Machine Learning and Deep Learning

Dataset: Bank Account Fraud (BAF) suite, NeurIPS 2022, file `Base.csv` (Kaggle [sgpjesus/bank-account-fraud-dataset-neurips-2022](https://kaggle.com/sgpjesus/bank-account-fraud-dataset-neurips-2022)) | **Tools:** Python, scikit-learn, LightGBM, Keras | **Platform:** Google Colab

Problem statement. When someone applies online for a bank account, the bank must decide in seconds whether the application is genuine or a **fraudulent account opening** (stolen identity, fake identity, money-mule account). Only about 1 in 100 applications is fraud, so checking everything by hand is impossible. The bank needs a model that gives every application a **fraud risk score** and sends the riskiest ones to an analyst first.

Business objective. Catch as much fraud as possible without flagging too many genuine customers (every false alarm delays a real customer and costs analyst time).

ML objective. Predict `fraud_bool` (1 = fraud, 0 = genuine) from the application details, the device and session behaviour and the recent "velocity" counts. Compare Logistic Regression, Random Forest and a tuned LightGBM model.

DL objective. Train a small Keras neural network (MLP) on the same data and check honestly whether it beats the machine learning models.

Dataset. 1,000,000 applications (32 columns) collected over **8 months** (`month` = 0 to 7). The file stores "missing" as `-1`, not as an empty cell, so we handle that first. `month` is used **only to split the data by time**, never as a model input. **No number in this notebook is typed in by hand: every count, rate and metric is calculated when you run the cells.**

How to run. Runtime > Run all. A full run should take roughly 20 to 40 minutes on a free Colab CPU. Set `FAST_MODE = True` in the next cell for a quick test (smaller sample, fewer trees, fewer epochs).

```
!pip install -q imbalanced-learn shap kagglehub

import os, glob, gc, time, random, warnings
import numpy as np, pandas as pd
import matplotlib.pyplot as plt, seaborn as sns
from IPython.display import display, Markdown

from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import RandomizedSearchCV, PredefinedSplit
from sklearn.inspection import permutation_importance
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score, roc_auc_score, average_precision_score,
                             confusion_matrix, roc_curve, precision_recall_curve)
from imblearn.under_sampling import RandomUnderSampler # resampling is only ever applied to TRAINING data
from imblearn.over_sampling import RandomOverSampler, SMOTE
from lightgbm import LGBMClassifier
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, regularizers, callbacks

warnings.filterwarnings("ignore")
SEED = 42
FAST_MODE = False # True = quick test run
SAMPLE_FRAC = 0.10 if FAST_MODE else 0.30 # share of the 1M rows used (1.0 = everything, needs more RAM and time)
N_TREES, N_ITER, EPOCHS = (60, 2, 3) if FAST_MODE else (300, 8, 40)
COST_FN, COST_FP = 500, 10 # ASSUMED costs: a missed fraud costs 500, a false alarm costs 10 (edited)
TARGET = "fraud_bool"
TRAIN_MONTHS, VAL_MONTHS, TEST_MONTHS = range(0, 5), [5], [6, 7] # train on the past, tune on month 5, test on future
random.seed(SEED); np.random.seed(SEED); keras.utils.set_random_seed(SEED)
sns.set_theme(style="whitegrid"); plt.rcParams.update({"figure.dpi": 100, "axes.titleweight": "bold"})
pd.set_option("display.max_columns", 60); pd.set_option("display.float_format", lambda x: f"{x:,.4f}")
BLUE, RED = "#4C72B0", "#C44E52"; PALETTE = {0: BLUE, 1: RED}

def interpret(*lines):
    """Short data-driven interpretation shown under an output."""
    display(Markdown("**Interpretation**\n\n" + "\n".join(f"- {l}" for l in lines)))
```

Phase 1: Data Understanding, Cleaning and EDA

We first load the data and check what is really in the file, because nothing is assumed. We then clean it, and explore it with charts to see what fraudulent applications look like.

```
def find_base_csv():
    """Download with kagglehub; if Kaggle cannot be reached, ask for a manual upload of Base.csv."""
    try:
        import kagglehub
        folder = kagglehub.dataset_download("sgpjesus/bank-account-fraud-dataset-neurips-2022")
        hits = glob.glob(os.path.join(folder, "**", "Base.csv"), recursive=True)
        if hits: return hits[0]
    except Exception as err:
        print("kagglehub failed:", repr(err))
    from google.colab import files
    return [k for k in files.upload() if k.lower().endswith(".csv")][0]

raw_df = pd.read_csv(find_base_csv())
N_RAW = len(raw_df)
df = raw_df.sample(frac=SAMPLE_FRAC, random_state=SEED).reset_index(drop=True) if SAMPLE_FRAC < 1 else raw_df.copy()
del raw_df; gc.collect()
print(f"Full file: {N_RAW:,} rows | working sample: {len(df):,} rows x {df.shape[1]} columns")
display(df.head(3)); df.info()

# Decide the role of every column from the real file
assert TARGET in df.columns and "month" in df.columns, f"Expected columns missing. Columns: {list(df.columns)}"
SENTINEL = [c for c in ["prev_address_months_count", "current_address_months_count", "bank_months_count",
                        "session_length_in_minutes", "device_distinct_emails_8w"] if c in df.columns] # -1 means missing
CAT_RAW = [c for c in df.columns if not pd.api.types.is_numeric_dtype(df[c])]
NUM_RAW = [c for c in df.columns if c not in CAT_RAW + [TARGET, "month"]]
BINARY_RAW = [c for c in NUM_RAW if df[c].nunique() == 2]
interpret(f"Target: **{TARGET}**. Categories: {CAT_RAW}. Binary flags: {BINARY_RAW}. Time column: **month** ({int} columns)
f"Columns where `-1` means 'missing': {SENTINEL} (plus negative values in `intended_balcon_amount`). The
```

Using Colab cache for faster access to the 'bank-account-fraud-dataset-neurips-2022' dataset.
Full file: 1,000,000 rows | working sample: 300,000 rows x 32 columns

	fraud_bool	income	name_email_similarity	prev_address_months_count	current_address_months_count	customer_ag
0	0	0.1000	0.2181	110	7	2
1	0	0.1000	0.3731	29	7	2
2	0	0.2000	0.5152	-1	64	2

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 300000 entries, 0 to 299999

Data columns (total 32 columns):

#	Column	Non-Null Count	Dtype
0	fraud_bool	300000 non-null	int64
1	income	300000 non-null	float64
2	name_email_similarity	300000 non-null	float64
3	prev_address_months_count	300000 non-null	int64
4	current_address_months_count	300000 non-null	int64
5	customer_age	300000 non-null	int64
6	days_since_request	300000 non-null	float64
7	intended_balcon_amount	300000 non-null	float64
8	payment_type	300000 non-null	object
9	zip_count_4w	300000 non-null	int64
10	velocity_6h	300000 non-null	float64
11	velocity_24h	300000 non-null	float64
12	velocity_4w	300000 non-null	float64
13	bank_branch_count_8w	300000 non-null	int64
14	date_of_birth_distinct_emails_4w	300000 non-null	int64
15	employment_status	300000 non-null	object
16	credit_risk_score	300000 non-null	int64
17	email_is_free	300000 non-null	int64
18	housing_status	300000 non-null	object
19	phone_home_valid	300000 non-null	int64
20	phone_mobile_valid	300000 non-null	int64
21	bank_months_count	300000 non-null	int64
22	has_other_cards	300000 non-null	int64
23	proposed_credit_limit	300000 non-null	float64
24	foreign_request	300000 non-null	int64
25	source	300000 non-null	object
26	session_length_in_minutes	300000 non-null	float64
27	device_os	300000 non-null	object
28	keep_alive_session	300000 non-null	int64
29	device_distinct_emails_8w	300000 non-null	int64
30	device_fraud_count	300000 non-null	int64
31	month	300000 non-null	int64

dtypes: float64(9), int64(18), object(5)

memory usage: 73.2+ MB

Interpretation

- Target: **fraud_bool**. Categories: ['payment_type', 'employment_status', 'housing_status', 'source', 'device_os']. Binary flags: ['email_is_free', 'phone_home_valid', 'phone_mobile_valid', 'has_other_cards', 'foreign_request', 'keep_alive_session']. Time column: **month** (0 to 7), used only for splitting.
- Columns where -1 means 'missing': ['prev_address_months_count', 'current_address_months_count', 'bank_months_count', 'session_length_in_minutes', 'device_distinct_emails_8w'] (plus negative values in intended_balcon_amount). There is no calendar date, so

```
# ---- Data quality checks ----
HIDDEN = SENTINEL + [c for c in ["intended_balcon_amount"] if c in df.columns]
def is_missing(frame, c):
    """BAF stores 'missing' as -1 (or a negative number in intended_balcon_amount), not as NaN."""
    return frame[c] < 0 if c == "intended_balcon_amount" else frame[c] == -1

miss = pd.DataFrame({"hidden_missing_pct": {c: is_missing(df, c).mean() * 100 for c in HIDDEN}}).sort_values("hidden_missing_pct")
display(miss.round(2))
n_nan, n_dup = int(df.isna().sum().sum()), int(df.duplicated().sum())
const_cols = [c for c in df.columns if df[c].nunique() <= 1]
checks = {"income outside [0, 1]": ((df["income"] < 0) | (df["income"] > 1)).sum(),
          "name_email_similarity outside [0, 1]": ((df["name_email_similarity"] < 0) | (df["name_email_similarity"] > 1)).sum(),
          "negative days_since_request": (df["days_since_request"] < 0).sum(),
          "negative velocity values": (df[["velocity_6h", "velocity_24h", "velocity_4w"]] < 0).any(axis=1).sum())

# ---- Cleaning ----
df_model = df.drop_duplicates().reset_index(drop=True) # duplicates in both train and test would leak
df_plot = df_model.copy() # plotting copy: hidden missing values become -1
for c in HIDDEN: df_plot[c] = df_plot[c].where(~is_missing(df_plot, c))

interpret(f"Most missing column: **{miss.index[0]**} ({miss['hidden_missing_pct'].iloc[0]:.1f}% of rows hold -1`
         f"Normal NaN cells: {n_nan}. Duplicate rows removed: {n_dup}. Constant columns (no information, dropped): {const_cols}.
         f"Impossible values: {{k: int(v) for k, v in checks.items()}}.",
         f"Clean table: **{len(df_model):}, rows**. Outliers (burst of applications, huge counts) are **kept**, I")
```

	hidden_missing_pct
intended_balcon_amount	74.3300
prev_address_months_count	71.2500
bank_months_count	25.4100
current_address_months_count	0.4300
session_length_in_minutes	0.2000
device_distinct_emails_8w	0.0400

Interpretation

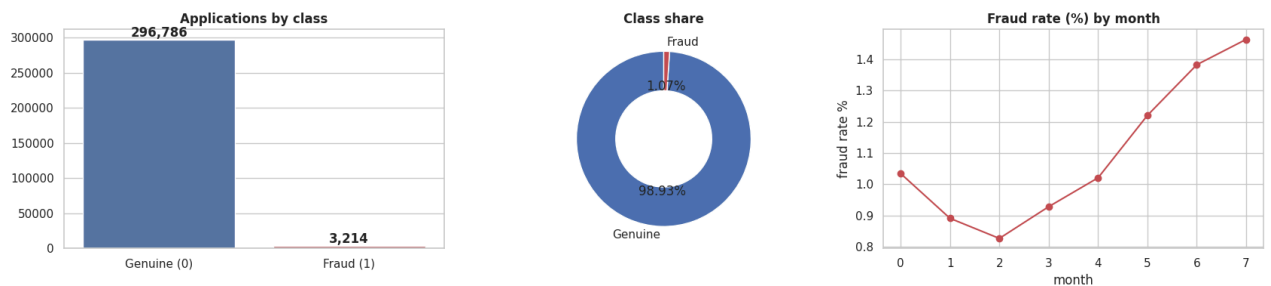
- Most missing column: **intended_balcon_amount** (74.3% of rows hold -1 or a negative value). A scaler or model would read -1 as a real number, so we convert it to NaN and keep an explicit missing flag.
- Normal NaN cells: 0. Duplicate rows removed: 0. Constant columns (no information, dropped later): ['device_fraud_count'].
- Impossible values: {'income outside [0, 1]': 0, 'name_email_similarity outside [0, 1]': 0, 'negative days_since_request': 0, 'negative velocity values': 14}.

1.1 Class imbalance and drift over time

Fraud is rare. If only a small share of applications are fraud, a model that always says "genuine" looks very accurate and catches nothing. We measure the imbalance first, and also check whether the fraud rate changes from month to month (this decides how we split the data).

```
counts = df_model[TARGET].value_counts().sort_index()
n_pos_all, n_neg_all = int(counts.get(1, 0)), int(counts.get(0, 0))
fraud_rate = df_model[TARGET].mean(); imbalance_ratio = n_neg_all / max(n_pos_all, 1)
by_month = df_model.groupby("month")[TARGET].agg(rate="mean", n_fraud="sum", n_apps="size"); by_month["rate"] *= 100

fig, ax = plt.subplots(1, 3, figsize=(17, 4))
sns.barplot(x=["Genuine (0)", "Fraud (1)"], y=counts.values, palette=[BLUE, RED], ax=ax[0])
for i, v in enumerate(counts.values): ax[0].text(i, v, f"{v:,}", ha="center", va="bottom", fontweight="bold")
ax[0].set_title("Applications by class")
ax[1].pie(counts.values, labels=["Genuine", "Fraud"], colors=[BLUE, RED], autopct="%1.2f%%", startangle=90, wedgeprops=dict(width=0.8))
ax[1].set_title("Class share")
ax[2].plot(by_month.index, by_month["rate"], marker="o", color=RED); ax[2].set(title="Fraud rate (%) by month", x=by_month.index)
plt.tight_layout(); plt.show()
interpret(f"{n_neg_all:,} genuine vs {n_pos_all:,} fraud applications: fraud is **{fraud_rate * 100:.2f}%** of the data.
Always predicting 'genuine' would be {100 - fraud_rate * 100:.2f}% accurate and catch **zero** fraud,
Fraud rate by month goes from {by_month['rate'].min():.2f}% to {by_month['rate'].max():.2f}%. Fraud behaviour changes over time, so we train on the past and test on the future (a time split) instead of shuffling randomly.")
```



Interpretation

- 296,786 genuine vs 3,214 fraud applications: fraud is **1.07%** of the data (about 92 genuine for every fraud).
- Always predicting 'genuine' would be 98.93% accurate and catch **zero** fraud, so we judge models with **recall, precision, F1, ROC-AUC and PR-AUC**, not accuracy.
- Fraud rate by month goes from 0.83% to 1.46%. Fraud behaviour changes over time, so we train on the past and test on the future (a time split) instead of shuffling randomly.

1.2 Missing fields and flags as fraud signals

Genuine customers usually have a history (previous address, previous bank account, device history). If that history is **missing**, the application may be riskier. We compare the fraud rate when a field is missing with when it is present, and the fraud rate for each 0/1 flag.

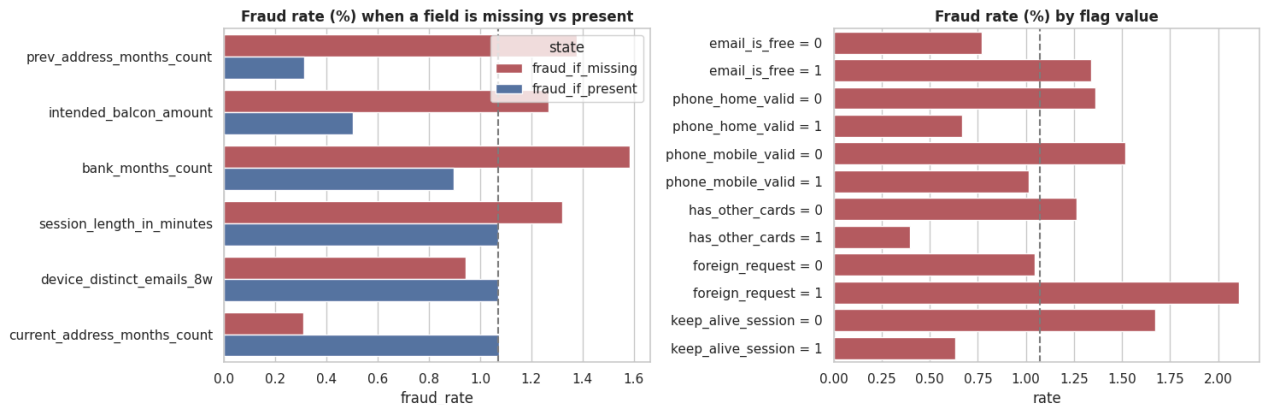
```
rows = []
for c in HIDDEN:
    m = is_missing(df_model, c)
```

```

if 0 < m.sum() < len(m): rows.append((c, m.mean() * 100, df_model.loc[m, TARGET].mean() * 100, df_model.loc[m, TARGET].mean() * 100 - df_model.loc[m, TARGET].mean() * 100))
gap = pd.DataFrame(rows, columns=["column", "pct_missing", "fraud_if_missing", "fraud_if_present"])
gap["gap"] = gap["fraud_if_missing"] - gap["fraud_if_present"]; gap = gap.sort_values("gap", ascending=False)

fig, ax = plt.subplots(1, 2, figsize=(15, 5))
g = gap.melt(id_vars="column", value_vars=["fraud_if_missing", "fraud_if_present"], var_name="state", value_name="rate")
sns.barplot(data=g, y="column", x="rate", hue="state", palette=[RED, BLUE], ax=ax[0]); ax[0].axvline(fraud_rate * 100, ls="--", c="grey")
ax[0].set_title("Fraud rate (%) when a field is missing vs present"); ax[0].set_ylabel("")
fl = pd.DataFrame([(f"{c} = {int(v)}", r * 100) for c in BINARY_RAW for v, r in df_model.groupby(c)[TARGET].mean().sort_values("rate").items()])
sns.barplot(data=fl, y="label", x="rate", color=RED, ax=ax[1]); ax[1].axvline(fraud_rate * 100, ls="--", c="grey")
ax[1].set_title("Fraud rate (%) by flag value"); ax[1].set_ylabel("")
plt.tight_layout(); plt.show()
top = gap.iloc[0]; fl_top = fl.sort_values("rate", ascending=False).iloc[0]
interpret(f"Biggest gap: {top['column']} has fraud rate {top['fraud_if_missing']:.2f}% when missing vs {top['fraud_if_present']:.2f}% when present.",
         f"Highest-risk flag value: {fl_top['label']} with fraud rate {fl_top['rate']:.2f}%.",
         "Where the red 'missing' bar is far above the blue one, *being missing* is itself a signal, so we create a missing flag for each of these columns in feature engineering.")

```



Interpretation

- Biggest gap: **prev_address_months_count** has fraud rate 1.38% when missing vs 0.31% when present (dashed line = overall 1.07%).
- Highest-risk flag value: **foreign_request = 1** with fraud rate 2.11%.
- Where the red 'missing' bar is far above the blue one, *being missing* is itself a signal, so we create a missing flag for each of these columns in feature engineering.

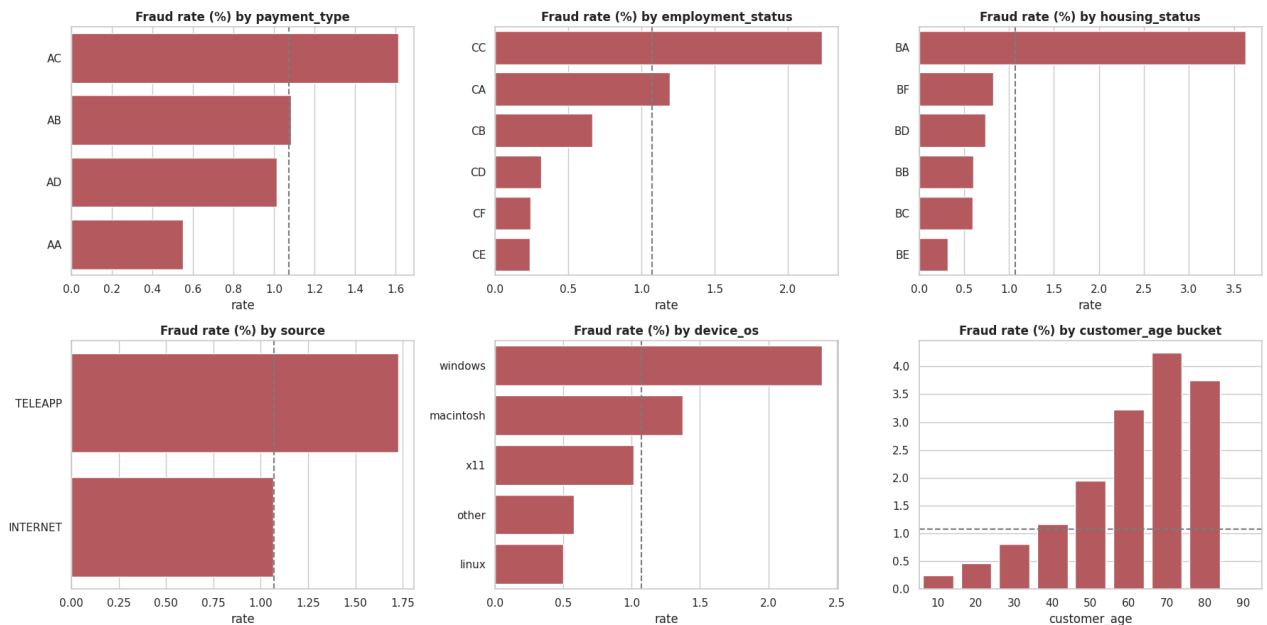
1.3 Categories and age

We check which category values and which age groups have more fraud (only groups with at least 200 applications, since tiny groups give unreliable rates).

```

cat_show = CAT_RAW[:5]
fig, axes = plt.subplots(2, 3, figsize=(18, 9)); axes = axes.ravel()
best_cat = []
for ax, c in zip(axes, cat_show):
    t = df_model.groupby(c)[TARGET].agg(count="size", rate="mean").reset_index()
    t = t[t["count"] >= 200].sort_values("rate", ascending=False); t["rate"] *= 100
    sns.barplot(data=t, y=c, x="rate", color=RED, ax=ax); ax.axvline(fraud_rate * 100, ls="--", c="grey"); ax.set_title(f"Fraud rate (%) by {c}")
    if len(t): best_cat.append(f"{c}: {t.iloc[0]['count']} at {t.iloc[0]['rate']:.1f}% (n={int(t.iloc[0]['count'])})")
age_t = df_model.groupby("customer_age")[TARGET].agg(count="size", rate="mean").reset_index()
sns.barplot(x=age_t.index.astype(int).astype(str), y=age_t.values, color=RED, ax=axes[len(cat_show)])
axes[len(cat_show)].axhline(fraud_rate * 100, ls="--", c="grey"); axes[len(cat_show)].set_title("Fraud rate (%) by age")
for ax in axes[len(cat_show) + 1:]: ax.axis("off")
plt.tight_layout(); plt.show()
interpret("Highest fraud rate among frequent groups: " + "; ".join(best_cat) + ".",
         f"Age bucket with the highest fraud rate: {age_t.idxmax()} ({age_t.max():.2f}%), lowest: {age_t.idxmin()} ({age_t.min():.2f}%).",
         "Age is a sensitive attribute. A model may use it as a shortcut and treat older or younger genuine customers differently.")

```



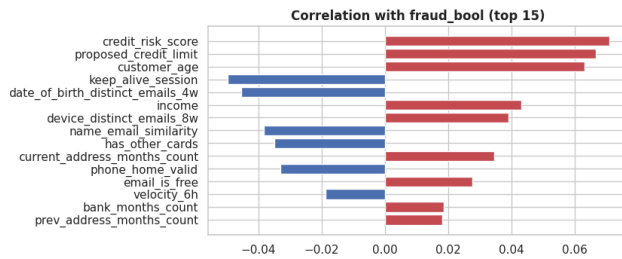
Interpretation

- Highest fraud rate among frequent groups: payment_type: **AC** at 1.6% (n=75,735); employment_status: **CC** at 2.2% (n=11,252); housing_status: **BA** at 3.6% (n=50,852); source: **TELEAPP** at 1.7% (n=2,087); device_os: **windows** at 2.4% (n=78,651).
- Age bucket with the highest fraud rate: **70** (4.25%), lowest: **90** (0.00%).
- Age is a sensitive attribute. A model may use it as a shortcut and treat older or younger genuine customers unfairly, so we run a fairness check in Phase 7.

1.4 Correlation

Correlation shows linear links only, so a weak value does not make a feature useless. In fraud data the signal is usually spread over many weak, non-linear patterns. The second chart shows features that say almost the same thing (redundant).

```
corr = df_plot[NUM_RAW + [TARGET]].corr()
tcorr = corr[TARGET].drop(TARGET).dropna().sort_values(key=np.abs, ascending=False)
top12 = tcorr.head(12).index.tolist()
fig, ax = plt.subplots(1, 2, figsize=(17, 6))
t15 = tcorr.head(15)[::-1]; ax[0].barh(t15.index, t15.values, color=[RED if v > 0 else BLUE for v in t15.values])
sns.heatmap(corr.loc[top12, top12], cmap="RdBu_r", center=0, annot=True, fmt=".2f", annot_kws={"size": 7}, cbar=fig)
plt.tight_layout(); plt.show()
cm = corr.drop(index=TARGET, columns=TARGET).abs()
pairs = cm.where(np.triu(np.ones(cm.shape), k=1).astype(bool)).stack(); pairs = pairs[pairs > 0.7].sort_values(ascending=False)
interpret(f"Strongest linear link with the target: **{tcorr.index[0]}** ({tcorr.iloc[0]:.3f}). Values this small ;
        "Redundant pairs (|r| > 0.7): " + (" ", ").join(f"{a} and {b} ({v:.2f})" for (a, b), v in pairs.head(3).it
```



Interpretation

- Strongest linear link with the target: **credit_risk_score** (0.071). Values this small are normal in fraud data.
- Redundant pairs (Irl > 0.7): none. This hurts Logistic Regression a little but not tree models.

Phase 2: Feature Engineering, Preprocessing and Split

Leakage rule. Every engineered feature is calculated **row by row from that application's own fields**, with no use of the target and no statistics from other rows. Anything that has to *learn* from data (median for filling gaps, scaling, one-hot categories) is fitted on the **training months only**.

Feature	Why it can help
<code><column>_missing</code> , <code>n_missing_flags</code>	Missing history can itself signal fraud
<code>credit_limit_to_income</code>	A very high credit limit compared with income is risky
<code>velocity_6h_to_24h</code> , <code>velocity_24h_to_4w</code>	Shows a sudden burst of applications (acceleration)
<code>address_months_total</code> , <code>address_to_age</code>	Living-stability of the applicant
<code>phones_valid</code>	How many phone numbers could be verified
<code>low_similarity_free_email</code>	Free email address that does not look like the applicant's name

Why a time split. A random split would mix months, so the model would be tested on the same period it learned from. Fraud changes over time, so we train on months 0 to 4, tune on month 5 and test once on months 6 and 7, like a real bank would (learn from the past, face the future). `month` itself is never a feature.

```

FLAG_COLS = [c + "_missing" for c in HIDDEN]
NEW_FEATS = ["credit_limit_to_income", "velocity_6h_to_24h", "velocity_24h_to_4w", "address_months_total", "address_to_age", "phones_valid", "low_similarity_free_email", "n_missing_flags"] + FLAG_COLS

def prepare(d):
    """Row-by-row cleaning and features only, so it is safe to run before the split."""
    d = d.copy()
    for c in HIDDEN:
        miss_c = is_missing(d, c)
        d[c + "_missing"] = miss_c.astype("int8")
        d[c] = d[c].where(~miss_c) # flag BEFORE the value is blanked
                                   # -1 becomes a real NaN
    d["credit_limit_to_income"] = d["proposed_credit_limit"] / (d["income"] + 1e-3)
    d["velocity_6h_to_24h"] = d["velocity_6h"] / (d["velocity_24h"].abs() + 1e-3)
    d["velocity_24h_to_4w"] = d["velocity_24h"] / (d["velocity_4w"].abs() + 1e-3)
    d["address_months_total"] = d["current_address_months_count"].fillna(0) + d["prev_address_months_count"]
    d["address_to_age"] = d["current_address_months_count"] / (d["customer_age"] * 12)
    d["phones_valid"] = d["phone_home_valid"] + d["phone_mobile_valid"]
    d["low_similarity_free_email"] = ((d["name_email_similarity"] < 0.2) & (d["email_is_free"] == 1)).astype("int8")
    d["n_missing_flags"] = d[FLAG_COLS].sum(axis=1)
    return d.drop(columns=[c for c in const_cols if c in d.columns]) # constant columns help nobody

df_p = prepare(df_model)
FEATURES = [c for c in df_p.columns if c not in [TARGET, "month"]]
CAT_FEATURES = [c for c in FEATURES if not pd.api.types.is_numeric_dtype(df_p[c])]
NUM_FEATURES = [c for c in FEATURES if c not in CAT_FEATURES]
```

```
# Do the new features carry signal? (how well one feature alone ranks fraud above genuine: 0.5 = no signal)
sig = pd.Series({c: roc_auc_score(df_p[TARGET], df_p[c].fillna(df_p[c].median())) for c in NEW_FEATS if df_p[c].n
sig = sig.reindex((sig - 0.5).abs().sort_values(ascending=False).index)
display(sig.head(8).to_frame())
interpret(f"{len(FEATURES)} candidate features: {len(NUM_FEATURES)} numeric (including flags) and {len(CAT_FEATURES)} categorical.
f"Strongest new feature on its own: **{sig.index[0]}** (AUC {sig.iloc[0]:.3f}; 0.5 would mean no signal)
```

	single_feature_AUC	
n_missing_flags	0.6545	
prev_address_months_count_missing	0.6030	
phones_valid	0.4052	
address_months_total	0.5941	
address_to_age	0.5891	
intended_balcon_amount_missing	0.5691	
low_similarity_free_email	0.5636	
bank_months_count_missing	0.5615	

Interpretation

- 43 candidate features: 38 numeric (including flags) and 5 categorical. fraud_bool and month are excluded.
- Strongest new feature on its own: **n_missing_flags** (AUC 0.654; 0.5 would mean no signal, below 0.5 means it points the other way). Whether the new features really help is checked later with feature importance.

```
def make_preprocessor():
    """Fresh, unfitted preprocessing: fill gaps with the median and scale numbers, one-hot encode categories."""
    return ColumnTransformer([("num", Pipeline([("imp", SimpleImputer(strategy="median")), ("sc", StandardScaler())]), ("cat", OneHotEncoder(handle_unknown="ignore", sparse_output=False), CAT_FEATURES)])

tr, va, te = [df_p["month"].isin(m) for m in (TRAIN_MONTHS, VAL_MONTHS, TEST_MONTHS)]
y_train, y_val, y_test = [df_p.loc[m, TARGET].values.astype(int) for m in (tr, va, te)]
prep = make_preprocessor()
X_train = prep.fit_transform(df_p.loc[tr, FEATURES]).astype("float32") # fitted on TRAIN months only
X_val, X_test = [prep.transform(df_p.loc[m, FEATURES]).astype("float32") for m in (va, te)]
FEAT_NAMES = [n.split("__", 1)[1] for n in prep.get_feature_names_out()]
n_pos, n_neg = int(y_train.sum()), int((y_train == 0).sum()); SPW = n_neg / n_pos # genuine applications per fraud

split_tbl = pd.DataFrame({"Rows": [len(y_train), len(y_val), len(y_test)], "Fraud": [y_train.sum(), y_val.sum(), y_test.sum()],
    "Genuine": [(y_train == 0).sum(), (y_val == 0).sum(), (y_test == 0).sum()],
    "Fraud rate %": [y_train.mean() * 100, y_val.mean() * 100, y_test.mean() * 100]},
    index=["Train (months 0-4)", "Validation (month 5)", "Test (months 6-7)"])
display(split_tbl.round(3))
BASELINE_ACC = accuracy_score(y_test, np.zeros_like(y_test))
interpret(f"Train: {len(y_train):,} applications ({n_pos:,} fraud). Validation: {len(y_val):,} ({int(y_val.sum()):,} fraud). Test: {len(y_test):,} ({int(y_test.sum()):,} fraud). The 'always genuine' baseline reaches {BASELINE_ACC * 100:.2f}% accuracy with recall 0. After preprocessing (fitted on train only) the model sees 64 columns. Validation is used to tune models and to choose thresholds. The test months are used only for the final scores.")
```

	Rows	Fraud	Genuine	Fraud rate %	
Train (months 0-4)	202559	1901	200658	0.9380	
Validation (month 5)	35881	438	35443	1.2210	
Test (months 6-7)	61560	875	60685	1.4210	

Interpretation

- Train: 202,559 applications (1,901 fraud). Validation: 35,881 (438 fraud). Test: 61,560 (875 fraud). With so few fraud cases in the test months, one extra miss moves recall by about 0.11 points, so read small differences with care.
- The 'always genuine' baseline reaches 98.58% accuracy with recall 0. After preprocessing (fitted on train only) the model sees 64 columns.
- Validation is used to tune models and to choose thresholds. The test months are used only for the final scores.

```
# ---- Evaluation helpers used for the rest of the notebook ----
def tpr_at_fpr(y, p, target=0.05):
    """Share of fraud caught if at most `target` (5%) of genuine applicants may be flagged."""
    fpr, tpr, _ = roc_curve(y, p); return float(tpr[fpr <= target].max())
def thr_at_fpr(y, p, target=0.05):
    fpr, tpr, thr = roc_curve(y, p); return float(thr[np.where(fpr <= target)[0][-1]])
def thr_best_f1(y, p):
    pr, rc, thr = precision_recall_curve(y, p); f1 = 2 * pr * rc / (pr + rc + 1e-12); return float(thr[np.nanargmax(f1)])
def confusion(y, prob, thr):
    tn, fp, fn, tp = confusion_matrix(y, (np.asarray(prob) >= thr).astype(int), labels=[0, 1]).ravel(); return tn, fp, fn, tp
def compute_metrics(y, prob, thr):
    pred = (np.asarray(prob) >= thr).astype(int); tn, fp, fn, tp = confusion(y, prob, thr)
    return {"Accuracy": accuracy_score(y, pred), "Precision": precision_score(y, pred, zero_division=0), "Recall": recall_score(y, pred, zero_division=0), "F1": f1_score(y, pred, zero_division=0), "ROC_AUC": roc_auc_score(y, prob), "PR_AUC": average_precision_score(y, prob), "TPR@5%FPR": tpr_at_fpr(y, prob), "TP": tp, "FP": fp, "FN": fn, "TN": tn}

RESULTS = {}
```

```
def add_result(name, kind, val_prob, test_prob, est=None):
    """Store scores and test metrics. The 0/1 threshold is chosen on the VALIDATION month (max F1), never on the
    thr = thr_best_f1(y_val, val_prob)
    m = compute_metrics(y_test, test_prob, thr); m["Threshold"] = thr
    RESULTS[name] = {"kind": kind, "val": np.asarray(val_prob).ravel(), "test": np.asarray(test_prob).ravel(), "
    print(f"{name:<20} Prec={m['Precision']:.4f} Rec={m['Recall']:.4f} F1={m['F1']:.4f} ROC-AUC={m['ROC_AUC']:.4f}
```

Phase 3: Handling Class Imbalance

We compare five ways to deal with the imbalance using the same fast LightGBM model: no handling, **class weights**, random undersampling, random oversampling and **SMOTE** (synthetic fraud examples). Every resampling step is fitted on the **training data only**, never on validation or test. The comparison is scored on the **validation month**, so the test months stay untouched.

```
def quick_lgbm(w=1.0):
    return LGBMClassifier(n_estimators=100, learning_rate=0.1, num_leaves=31, scale_pos_weight=w, n_jobs=-1, random_state=SEED)

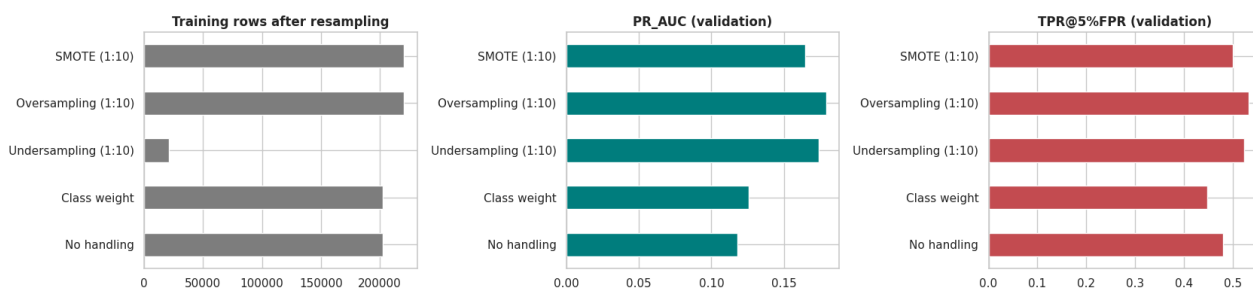
strategies = {"No handling": (None, 1.0), "Class weight": (None, SPW),
              "Undersampling (1:10)": (RandomUnderSampler(sampling_strategy=0.1, random_state=SEED), 1.0),
              "Oversampling (1:10)": (RandomOverSampler(sampling_strategy=0.1, random_state=SEED), 1.0),
              "SMOTE (1:10)": (SMOTE(sampling_strategy=0.1, random_state=SEED), 1.0)}

rows = []
for name, (sampler, w) in strategies.items():
    Xr, yr = (X_train, y_train) if sampler is None else sampler.fit_resample(X_train, y_train) # TRAIN ONLY
    pv = quick_lgbm(w).fit(Xr, yr).predict_proba(X_val)[: , 1]
    rows.append({"Strategy": name, "train_rows": len(yr), "PR_AUC": average_precision_score(y_val, pv), "ROC_AUC": roc_auc_score(y_val, pv)})
imb = pd.DataFrame(rows).set_index("Strategy"); display(imb.round(4))

fig, ax = plt.subplots(1, 3, figsize=(17, 4))
for a, (col, color) in zip(ax, [("train_rows", "grey"), ("PR_AUC", "teal"), ("TPR@5%FPR", "red")]): imb[col].plot.bar(ax=ax[a], color=color)
plt.tight_layout(); plt.show()

best_imb = imb["PR_AUC"].idxmax(); gap_cw = imb["PR_AUC"].max() - imb.loc["Class weight", "PR_AUC"]
imb_note = ("class weights had the best validation PR-AUC" if gap_cw <= 1e-12 else
            f"'{best_imb}' was higher than class weights by {gap_cw:.4f} PR-AUC" + (" (within noise, so the simple
interpret(f"Best validation PR-AUC: **{best_imb}** ({imb['PR_AUC'].max():.4f}). Class weights reach {imb.loc['Class weight', 'PR_AUC']:.4f}.
f"Decision: the main models use **class weights** ('class_weight' / 'scale_pos_weight'). Note: {imb_note}
"Undersampling throws most genuine rows away, and oversampling or SMOTE adds copies or synthetic points")
```

	train_rows	PR_AUC	ROC_AUC	TPR@5%FPR
Strategy				
No handling	202559	0.1180	0.8722	0.4795
Class weight	202559	0.1260	0.8345	0.4475
Undersampling (1:10)	20911	0.1741	0.8779	0.5228
Oversampling (1:10)	220723	0.1792	0.8782	0.5320
SMOTE (1:10)	220723	0.1648	0.8785	0.5000



Interpretation

- Best validation PR-AUC: **Oversampling (1:10)** (0.1792). Class weights reach 0.1260; SMOTE reaches 0.1648; no handling reaches 0.1180.
- Decision: the main models use **class weights** (class_weight / scale_pos_weight). Note: 'Oversampling (1:10)' was higher than class weights by 0.0533 PR-AUC, but class weights were kept because they leave the data untouched and are simple to explain.
- Undersampling throws most genuine rows away, and oversampling or SMOTE adds copies or synthetic points, which can look unrealistic. Class weights only tell the model that a missed fraud matters more.

Phase 4: Machine Learning Track

We compare three models, all trained on the training months only:

1. **Logistic Regression**, the simple, explainable baseline.
2. **Random Forest**, many decision trees voting together.
3. **LightGBM**, gradient boosting (each new tree fixes the errors of the earlier ones), very strong on tables like this one. We then **tune** it with `RandomizedSearchCV`, optimising **PR-AUC** (not accuracy), and compare it before and after tuning.

Every model is scored on the same untouched test months. The 0/1 threshold of each model is picked on the validation month (the value that gives the best F1). Section 6 then looks at better thresholds.

```
models = {"Logistic Regression": LogisticRegression(C=0.1, class_weight="balanced", max_iter=2000),
          "Random Forest": RandomForestClassifier(n_estimators=N_TREES // 2, max_depth=14, min_samples_leaf=10,
                                                  class_weight="balanced_subsample", n_jobs=-1, random_state=SEE),
          "LightGBM": LGBMClassifier(n_estimators=N_TREES, learning_rate=0.05, num_leaves=31, min_child_samples=
                                   colsample_bytree=0.8, reg_lambda=5, scale_pos_weight=SPW, n_jobs=-1, random
for name, est in models.items():
    est.fit(X_train, y_train)
    add_result(name, "ML", est.predict_proba(X_val)[: , 1], est.predict_proba(X_test)[: , 1], est)
b = pd.DataFrame({n: RESULTS[n]["metrics"] for n in RESULTS}).T.astype(float)
interpret(f"Accuracy is not shown here on purpose: the 'always genuine' baseline already gets {BASELINE_ACC:.3f}
f"Best baseline by PR-AUC: **{b['PR_AUC'].idxmax()}**. Precision is low for every model because even a
```

Logistic Regression	Prec=0.2166	Rec=0.2297	F1=0.2230	ROC-AUC=0.8790	PR-AUC=0.1540	TPR@5%FPR=0.4789	TP=201 FP=7
Random Forest	Prec=0.1901	Rec=0.2023	F1=0.1960	ROC-AUC=0.8662	PR-AUC=0.1275	TPR@5%FPR=0.4606	TP=177 FP=7
LightGBM	Prec=0.2538	Rec=0.1737	F1=0.2062	ROC-AUC=0.8638	PR-AUC=0.1517	TPR@5%FPR=0.4811	TP=152 FP=4

Interpretation

- Accuracy is not shown here on purpose: the 'always genuine' baseline already gets 0.986. Recall ranges from 0.174 to 0.230 and PR-AUC from 0.127 to 0.154 (a model with no skill would get a PR-AUC of about 0.014).
- Best baseline by PR-AUC: **Logistic Regression**. Precision is low for every model because even a small false-alarm rate on a huge genuine majority creates many alerts. This is normal for rare-event detection.

```
# Tune LightGBM: random search where every candidate trains on the TRAINING months and is scored on the VALIDATION months
X_tv, y_tv = np.vstack([X_train, X_val]), np.concatenate([y_train, y_val])
cv_split = PredefinedSplit(np.r_[np.full(len(y_train), -1), np.zeros(len(y_val), dtype=int)])
space = {"n_estimators": [200, 300, 450], "num_leaves": [15, 31, 63], "learning_rate": [0.03, 0.05, 0.1], "min_child
        "subsample": [0.6, 0.8, 1.0], "colsample_bytree": [0.5, 0.7, 0.9], "reg_lambda": [0, 5, 20], "scale_pos_w
search = RandomizedSearchCV(LGBMClassifier(subsample_freq=1, random_state=SEED, n_jobs=-1, verbose=-1), space, n
        scoring={"pr_auc": "average_precision", "recall": "recall", "precision": "precision"},
t0 = time.time(); search.fit(X_tv, y_tv)
bi = int(np.argmax(search.cv_results_["rank_test_pr_auc"])); best_params = search.cv_results_["params"][bi]
tuned = LGBMClassifier(subsample_freq=1, random_state=SEED, n_jobs=-1, verbose=-1).set_params(**best_params).fit(X_train, y_train)
add_result("LightGBM (Tuned)", "ML", tuned.predict_proba(X_val)[: , 1], tuned.predict_proba(X_test)[: , 1], tuned)
before, after = RESULTS["LightGBM"]["metrics"], RESULTS["LightGBM (Tuned)"]["metrics"]
print("Best parameters:", {k: (round(float(v), 4) if isinstance(v, (float, np.floating)) else v) for k, v in best
d = after["PR_AUC"] - before["PR_AUC"]
interpret(f"Validation-month results of the best setting: PR-AUC {search.cv_results_['mean_test_pr_auc'][bi]:.4f}
f"Test PR-AUC moved from {before['PR_AUC']:.4f} to {after['PR_AUC']:.4f} ({d:+.4f}); recall from {before['recall']:.4f} to {after['recall']:.4f} ({d:+.4f}); precision from {before['precision']:.4f} to {after['precision']:.4f} ({d:+.4f}); tuning helped."
" Tuning does not always help. A setting that is best on month 5 may not be best on months 6 and 7 because fraud patterns drift, and with few fraud cases small changes can be noise.
```

LightGBM (Tuned)	Prec=0.2147	Rec=0.2811	F1=0.2434	ROC-AUC=0.8788	PR-AUC=0.1667	TPR@5%FPR=0.4949	TP=246 FP=9
------------------	-------------	------------	-----------	----------------	---------------	------------------	-------------

Best parameters: {'subsample': 1.0, 'scale_pos_weight': 10.2739, 'reg_lambda': 5, 'num_leaves': 15, 'n_estimators': 300}

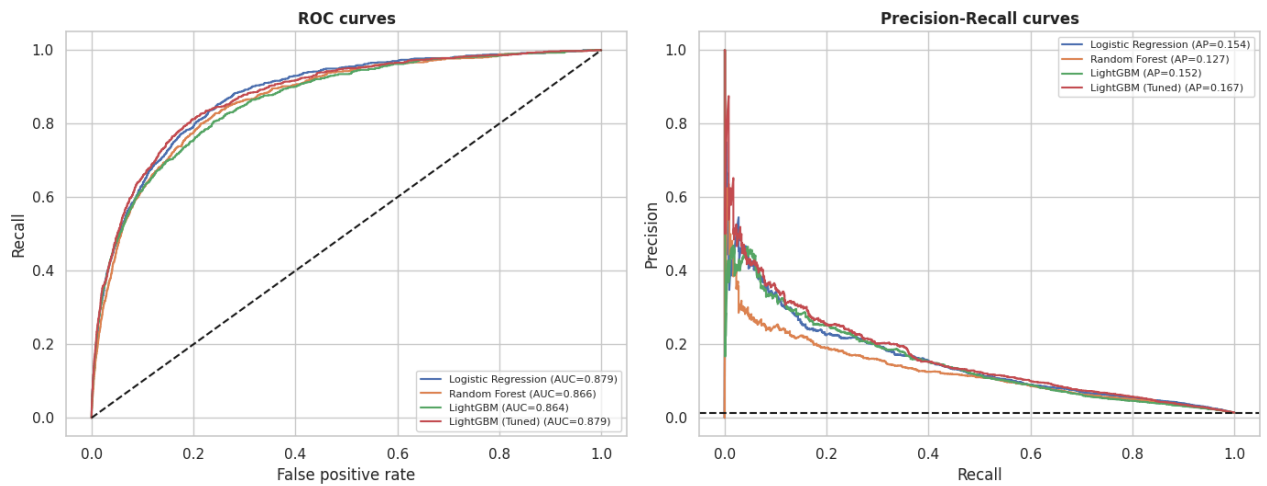
Interpretation

- Validation-month results of the best setting: PR-AUC 0.1767, recall 0.2443, precision 0.2399.
- Test PR-AUC moved from 0.1517 to 0.1667 (+0.0150); recall from 0.174 to 0.281. Tuning helped.
- Tuning does not always help. A setting that is best on month 5 may not be best on months 6 and 7 because fraud patterns drift, and with few fraud cases small changes can be noise.

4.1 ROC and Precision-Recall curves

The ROC curve is better when it hugs the top-left corner. The **Precision-Recall curve** is the stricter test for rare fraud: the dashed line shows what a random model would get (it equals the fraud rate). The huge number of genuine applications makes even a weak model look good on ROC, so the PR curve is the more honest of the two.

```
def plot_curves(names, title=""):
    fig, ax = plt.subplots(1, 2, figsize=(14, 5.5))
    for n in names:
        f, t, _ = roc_curve(y_test, RESULTS[n]["test"]); ax[0].plot(f, t, label=f"{n} (AUC={RESULTS[n]['metrics']['AUC']:.4f})")
        p, r, _ = precision_recall_curve(y_test, RESULTS[n]["test"]); ax[1].plot(r, p, label=f"{n} (AP={RESULTS[n]['metrics']['AP']:.4f})")
    ax[0].plot([0, 1], [0, 1], "k--"); ax[0].set(title="ROC curves" + title, xlabel="False positive rate", ylabel="True positive rate")
    ax[1].axhline(y_test.mean(), c="k", ls="--"); ax[1].set(title="Precision-Recall curves" + title, xlabel="Recall", ylabel="Precision")
    plt.tight_layout(); plt.show()
plot_curves(list(RESULTS))
interpret("A curve that stays above the others over the whole range is the better ranker; crossing curves mean the models disagree on the best threshold."
f"The random-model line in the PR plot sits at {y_test.mean():.3f}. Curves far above it show real learning.")
```



Interpretation

- A curve that stays above the others over the whole range is the better ranker; crossing curves mean the best model depends on how many alerts the bank can handle.
- The random-model line in the PR plot sits at 0.014. Curves far above it show real learning. Most of the difference between models appears at the left of the PR plot, where the very riskiest applications are.

Phase 5: Deep Learning Track (Keras Neural Network)

A neural network needs scaled, numeric input. Our preprocessing already gives that (numbers scaled, categories one-hot encoded, gaps filled), and the validation month is kept for early stopping, so no extra split is needed. The test months stay untouched.

Architecture (chosen for this data: many rows but very few fraud cases and only a few dozen inputs, so a small and regularised network; the exact row, fraud and input counts are printed by the cells above and below).

Part	Choice	Reason
Input	One neuron per column	Takes the preprocessed features
Hidden layers	128, 64, 32 neurons, ReLU	Learns feature combinations while narrowing step by step; ReLU trains fast
Batch normalisation	After every hidden layer	Stabilises training with mixed binary and continuous inputs
Dropout	0.3, 0.3, 0.2	Randomly switches neurons off so the network cannot memorise
Output	1 neuron, sigmoid	Fraud probability between 0 and 1
Loss / optimizer	Binary cross-entropy with class weights / Adam (lr 0.001)	Standard yes-no loss; the fraud weight is the square root of the imbalance ratio (the full ratio make
Batch size / epochs	1024 / up to <code>EPOCHS</code>	Large batches make sure every batch contains some fraud
Callbacks	EarlyStopping on validation PR-AUC, ReduceLROnPlateau	Stop when learning stops helping; lower the learning rate when progress stalls

```
def build_mlp(input_dim):
    inp = keras.Input(shape=(input_dim,)); x = inp
    for units, drop in [(128, 0.3), (64, 0.3), (32, 0.2)]:
        x = layers.Dense(units, kernel_regularizer=regularizers.l2(1e-5))(x)
        x = layers.BatchNormalization()(x); x = layers.Activation("relu")(x); x = layers.Dropout(drop)(x)
    model = keras.Model(inp, layers.Dense(1, activation="sigmoid")(x), name="fraud_mlp")
    model.compile(optimizer=keras.optimizers.Adam(1e-3), loss="binary_crossentropy",
                  metrics=[keras.metrics.Precision(name="precision"), keras.metrics.Recall(name="recall"),
                           keras.metrics.AUC(name="roc_auc"), keras.metrics.AUC(curve="PR", name="pr_auc")])

    return model

keras.utils.set_random_seed(SEED)
mlp = build_mlp(X_train.shape[1])
class_weight = {0: 1.0, 1: float(np.sqrt(SPW))}
print("Class weights:", class_weight, "| fraud cases in train/validation/test:", int(y_train.sum()), int(y_val.sum()))
mlp.summary()
```

Class weights: {0: 1.0, 1: 10.27394369217611} | fraud cases in train/validation/test: 1901 438 875
 Model: "fraud_mlp"

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 64)	0
dense (Dense)	(None, 128)	8,320
batch_normalization (BatchNormalization)	(None, 128)	512
activation (Activation)	(None, 128)	0
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 64)	8,256
batch_normalization_1 (BatchNormalization)	(None, 64)	256
activation_1 (Activation)	(None, 64)	0
dropout_1 (Dropout)	(None, 64)	0
dense_2 (Dense)	(None, 32)	2,080
batch_normalization_2 (BatchNormalization)	(None, 32)	128
activation_2 (Activation)	(None, 32)	0
dropout_2 (Dropout)	(None, 32)	0
dense_3 (Dense)	(None, 1)	33

Total params: 19,585 (76.50 KB)
Trainable params: 19,137 (74.75 KB)
Non-trainable params: 448 (1.75 KB)

```
cb = [callbacks.EarlyStopping(monitor="val_pr_auc", mode="max", patience=8, restore_best_weights=True, verbose=1),
      callbacks.ReduceLROnPlateau(monitor="val_pr_auc", mode="max", factor=0.5, patience=3, min_lr=1e-5, verbose=1)]
history = mlp.fit(X_train, y_train, validation_data=(X_val, y_val), epochs=EPOCHS, batch_size=1024, class_weight=class_weight)
hist = pd.DataFrame(history.history); print(f"Stopped after {len(hist)} epochs")
```

Epoch 1/40
 198/198 - 9s - 44ms/step - loss: 0.4827 - pr_auc: 0.0159 - precision: 0.0182 - recall: 0.2656 - roc_auc: 0.6789
 Epoch 2/40
 198/198 - 3s - 14ms/step - loss: 0.2690 - pr_auc: 0.0606 - precision: 0.1002 - recall: 0.1883 - roc_auc: 0.8292
 Epoch 3/40
 198/198 - 3s - 17ms/step - loss: 0.2478 - pr_auc: 0.0723 - precision: 0.1145 - recall: 0.2436 - roc_auc: 0.8494
 Epoch 4/40
 198/198 - 3s - 17ms/step - loss: 0.2417 - pr_auc: 0.0811 - precision: 0.1131 - recall: 0.2567 - roc_auc: 0.8602
 Epoch 5/40
 198/198 - 3s - 14ms/step - loss: 0.2344 - pr_auc: 0.0931 - precision: 0.1289 - recall: 0.2704 - roc_auc: 0.8682
 Epoch 6/40
 198/198 - 3s - 14ms/step - loss: 0.2356 - pr_auc: 0.0812 - precision: 0.1157 - recall: 0.2567 - roc_auc: 0.8702
 Epoch 7/40
 198/198 - 3s - 15ms/step - loss: 0.2330 - pr_auc: 0.0893 - precision: 0.1244 - recall: 0.2525 - roc_auc: 0.8719
 Epoch 8/40

Epoch 8: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.
 198/198 - 4s - 19ms/step - loss: 0.2297 - pr_auc: 0.0878 - precision: 0.1213 - recall: 0.2609 - roc_auc: 0.8793
 Epoch 9/40
 198/198 - 3s - 14ms/step - loss: 0.2274 - pr_auc: 0.0923 - precision: 0.1259 - recall: 0.2688 - roc_auc: 0.8821
 Epoch 10/40
 198/198 - 3s - 14ms/step - loss: 0.2263 - pr_auc: 0.0941 - precision: 0.1312 - recall: 0.2867 - roc_auc: 0.8833
 Epoch 11/40
 198/198 - 3s - 14ms/step - loss: 0.2245 - pr_auc: 0.0955 - precision: 0.1300 - recall: 0.2741 - roc_auc: 0.8865
 Epoch 12/40
 198/198 - 4s - 20ms/step - loss: 0.2227 - pr_auc: 0.0980 - precision: 0.1334 - recall: 0.2735 - roc_auc: 0.8877
 Epoch 13/40

Epoch 13: ReduceLROnPlateau reducing learning rate to 0.0002500000118743628.
 198/198 - 4s - 20ms/step - loss: 0.2230 - pr_auc: 0.0922 - precision: 0.1213 - recall: 0.2630 - roc_auc: 0.8882
 Epoch 14/40
 198/198 - 3s - 14ms/step - loss: 0.2220 - pr_auc: 0.0970 - precision: 0.1255 - recall: 0.2662 - roc_auc: 0.8898
 Epoch 15/40
 198/198 - 3s - 15ms/step - loss: 0.2199 - pr_auc: 0.0980 - precision: 0.1355 - recall: 0.2772 - roc_auc: 0.8926
 Epoch 16/40
 198/198 - 4s - 20ms/step - loss: 0.2198 - pr_auc: 0.0990 - precision: 0.1305 - recall: 0.2820 - roc_auc: 0.8923
 Epoch 17/40
 198/198 - 3s - 14ms/step - loss: 0.2164 - pr_auc: 0.1022 - precision: 0.1307 - recall: 0.2730 - roc_auc: 0.8975
 Epoch 18/40

Epoch 18: ReduceLROnPlateau reducing learning rate to 0.0001250000059371814.
 198/198 - 3s - 14ms/step - loss: 0.2172 - pr_auc: 0.1020 - precision: 0.1375 - recall: 0.3014 - roc_auc: 0.8963
 Epoch 19/40
 198/198 - 3s - 14ms/step - loss: 0.2178 - pr_auc: 0.1021 - precision: 0.1306 - recall: 0.2804 - roc_auc: 0.8955
 Epoch 20/40
 198/198 - 4s - 20ms/step - loss: 0.2162 - pr_auc: 0.1032 - precision: 0.1326 - recall: 0.2720 - roc_auc: 0.8984

Epoch 21/40

Epoch 21: ReduceLRonPlateau reducing learning rate to 6.25000029685907e-05.

198/198 - 4s - 20ms/step - loss: 0.2168 - pr_auc: 0.0993 - precision: 0.1326 - recall: 0.2709 - roc_auc: 0.8974

Epoch 22/40

198/198 - 3s - 14ms/step - loss: 0.2155 - pr_auc: 0.1073 - precision: 0.1385 - recall: 0.2893 - roc_auc: 0.8987

Epoch 23/40

198/198 - 3s - 14ms/step - loss: 0.2153 - pr_auc: 0.1019 - precision: 0.1307 - recall: 0.2777 - roc_auc: 0.8980

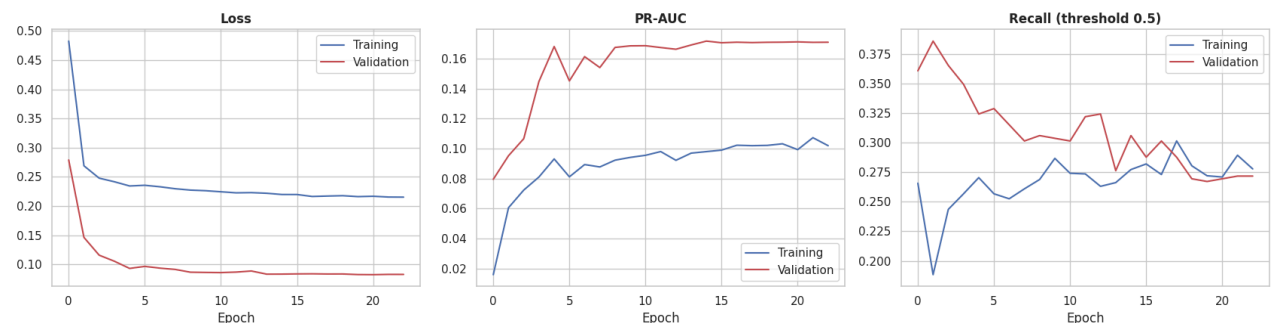
Epoch 23: early stopping

Restoring model weights from the end of the best epoch: 15.

Stopped after 23 epochs

```
fig, ax = plt.subplots(1, 3, figsize=(17, 4.5))
for a, (m, t) in zip(ax, [("loss", "Loss"), ("pr_auc", "PR-AUC"), ("recall", "Recall (threshold 0.5)")):
    a.plot(hist[m], label="Training", c=BLUE); a.plot(hist[f"val_{m}"], label="Validation", c=RED); a.set(title=t)
plt.tight_layout(); plt.show()

dl_val = mlp.predict(X_val, batch_size=8192, verbose=0).ravel(); dl_test = mlp.predict(X_test, batch_size=8192, v
DL_NAME = "Deep Learning (MLP)"; add_result(DL_NAME, "DL", dl_val, dl_test, mlp)
be = int(hist["val_pr_auc"].idxmax())
interpret(f"Best epoch by validation PR-AUC: {be + 1} (validation {hist['val_pr_auc'].max():.3f}, training {hist[
    "**Overfitting check:** overfitting shows as training metrics that keep improving while validation metr
```



Deep Learning (MLP) Prec=0.2247 Rec=0.2583 F1=0.2403 ROC-AUC=0.8788 PR-AUC=0.1585 TPR@5%FPR=0.4834 | TP=226 FP=7

Interpretation

- Best epoch by validation PR-AUC: 15 (validation 0.172, training 0.098, gap -0.074). At the last epoch, validation loss minus training loss is -0.132.
- Overfitting check:** overfitting shows as training metrics that keep improving while validation metrics stall or get worse. Early stopping with restored best weights, dropout and L2 limit it. Part of the gap is also expected because the validation month is *later* than the training months (fraud drifts), and dropout is active only during training.

Phase 6: Evaluation, Model Selection and Threshold

Reading the confusion matrix here. TP = fraud caught. TN = genuine application approved. FP = genuine customer wrongly flagged (delay, friction, analyst time). FN = fraud missed (direct financial loss). **False negatives are usually the costlier error**, so we pay close attention to recall while keeping the alert volume manageable.

We now put all models side by side. The **final model** is the one with the highest **PR-AUC** (the best threshold-free measure for rare fraud). We also show which model is cheapest under the assumed costs `COST_FN` and `COST_FP`, which are illustrative, not real bank numbers.

```
comparison = pd.DataFrame({n: r["metrics"] for n, r in RESULTS.items()}).T.astype(float)
comparison.insert(0, "Type", [r["kind"] for r in RESULTS.values()])
comparison["Expected_cost"] = comparison["FN"] * COST_FN + comparison["FP"] * COST_FP
NO_MODEL_COST = float(y_test.sum()) * COST_FN # approve everybody: every fraud gets through
comparison = comparison.sort_values("PR_AUC", ascending=False)
show_cols = ["Type", "Precision", "Recall", "F1", "ROC_AUC", "PR_AUC", "TPR@5%FPR", "TP", "FP", "FN", "Expected_c
display(comparison[show_cols].style.format({c: "{:.4f}" for c in ["Precision", "Recall", "F1", "ROC_AUC", "PR_AUC
    .background_gradient(subset=["Recall", "F1", "PR_AUC"], cmap="Greens"))

names = list(comparison.index)
fig, axes = plt.subplots(1, len(names), figsize=(4.1 * len(names), 3.8))
for a, n in zip(np.atleast_1d(axes), names):
    m = RESULTS[n]["metrics"]; sns.heatmap([m["TN"], m["FP"], m["FN"], m["TP"]], annot=True, fmt=".0f", cmap=
        xticklabels=["Pred genuine", "Pred fraud"], yticklabels=["Actual genui

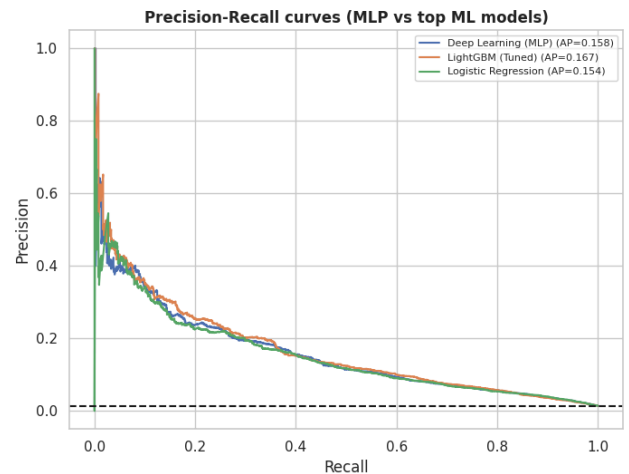
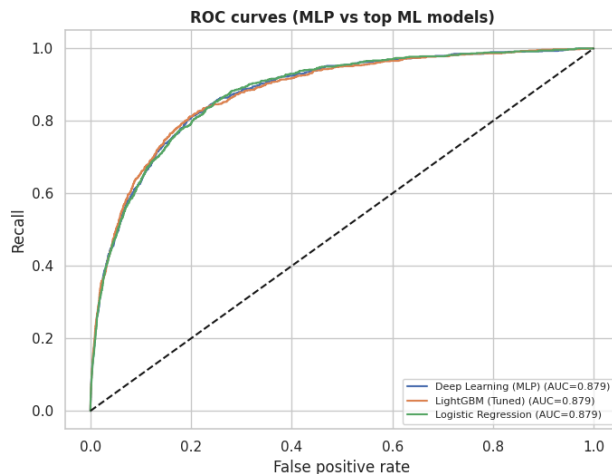
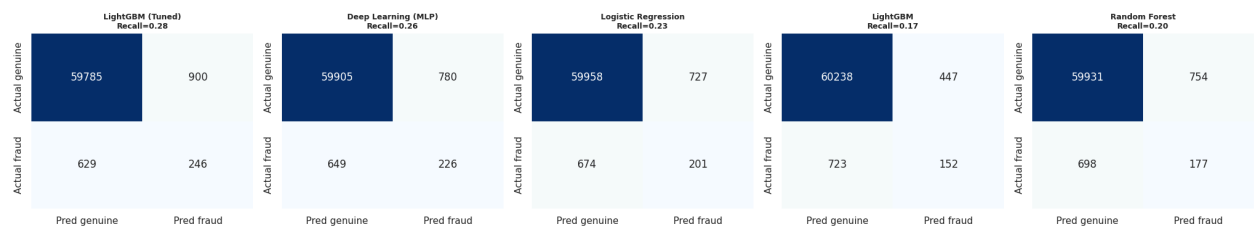
plt.tight_layout(); plt.show()
plot_curves([DL_NAME] + [n for n in names if RESULTS[n]["kind"] == "ML"][:2], " (MLP vs top ML models)")
```

```

best_ml_name = comparison[comparison['type'] == 'ML'].idxmax()
FINAL_NAME = comparison["PR_AUC"].idxmax()
best_business = comparison.sort_values(["Expected_cost", "PR_AUC"], ascending=[True, False]).index[0]
fm, bm, dm = comparison.loc[FINAL_NAME], comparison.loc[best_ml_name], comparison.loc[DL_NAME]
interpret(f"*Final model: {FINAL_NAME}* (highest PR-AUC {fm['PR_AUC']:.4f}); at its validation-chosen threshold
f"Best recall: {comparison['Recall'].idxmax()} ({comparison['Recall'].max():.3f}). Best precision: {com
f"Best ML model: {best_ml_name} (PR-AUC {bm['PR_AUC']:.4f}). Neural network: PR-AUC {dm['PR_AUC']:.4f},
"Close scores can swap places on another split because the test months have few fraud cases. Do not ove

```

	Type	Precision	Recall	F1	ROC_AUC	PR_AUC	TPR@5%FPR	TP	FP	FN	Expected_co
LightGBM (Tuned)	ML	0.2147	0.2811	0.2434	0.8788	0.1667	0.4949	246.000000	900.000000	629.000000	323,5
Deep Learning (MLP)	DL	0.2247	0.2583	0.2403	0.8788	0.1585	0.4834	226.000000	780.000000	649.000000	332,3
Logistic Regression	ML	0.2166	0.2297	0.2230	0.8790	0.1540	0.4789	201.000000	727.000000	674.000000	344,2
LightGBM	ML	0.2538	0.1737	0.2062	0.8638	0.1517	0.4811	152.000000	447.000000	723.000000	365,5
Random Forest	ML	0.1901	0.2023	0.1960	0.8662	0.1275	0.4606	177.000000	754.000000	698.000000	356,5



Interpretation

- **Final model: LightGBM (Tuned)** (highest PR-AUC 0.1667); at its validation-chosen threshold it has recall 0.281, precision 0.215, F1 0.243.
- Best recall: LightGBM (Tuned) (0.281). Best precision: LightGBM (0.254). Best F1: LightGBM (Tuned) (0.243). Cheapest under the assumed costs at these thresholds: **LightGBM (Tuned)**.
- Best ML model: LightGBM (Tuned) (PR-AUC 0.1667). Neural network: PR-AUC 0.1585, which did **not** beat the best ML model. On table data like this, boosted trees are often as good as or better than a neural network, so either result is normal.
- Close scores can swap places on another split because the test months have few fraud cases. Do not over-read tiny gaps (about 0.01 PR-AUC or less).

6.1 Threshold optimisation

A model outputs a score, and the threshold decides when it becomes a "fraud" flag. A **lower threshold** catches more fraud (higher recall) but creates more false alarms; a **higher threshold** gives fewer false alarms but lets more fraud through. We choose the threshold on the **validation month**, then apply it once to the test months. Choosing it on the test months would make the result look better than it is.

```

s_val, s_test = RESULTS[FINAL_NAME]["val"], RESULTS[FINAL_NAME]["test"]

def sweep(yt, p):

```

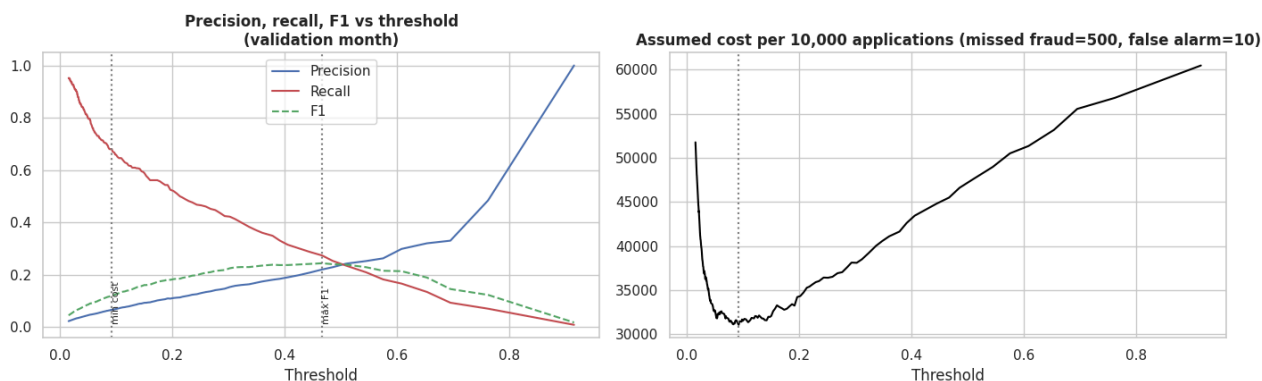
```

rows = []
for t in np.unique(np.quantile(p, np.linspace(0.5, 0.9999, 300))): # candidate thresholds from the score
    tn, fp, fn, tp = confusion(yt, p, t)
    pr, rc = tp / max(tp + fp, 1), tp / max(tp + fn, 1)
    rows.append({"threshold": t, "precision": pr, "recall": rc, "f1": 2 * pr * rc / max(pr + rc, 1e-12), "cost":
    return pd.DataFrame(rows)
sw = sweep(y_val, s_val)
thr_f1 = float(sw.loc[sw["f1"].idxmax(), "threshold"])
thr_cost = float(np.median(sw.loc[sw["cost"] == sw["cost"].min(), "threshold"]))
thr_fpr5, thr_fpr1 = thr_at_fpr(y_val, s_val, 0.05), thr_at_fpr(y_val, s_val, 0.01)
RECOMMENDED_THR = thr_cost

fig, ax = plt.subplots(1, 2, figsize=(14, 4.5))
ax[0].plot(sw["threshold"], sw["precision"], label="Precision", c=BLUE); ax[0].plot(sw["threshold"], sw["recall"],
for t, l in [(thr_f1, "max F1"), (thr_cost, "min cost")]: ax[0].axvline(t, c="grey", ls=":"); ax[0].text(t, 0.02,
ax[0].set(title="Precision, recall, F1 vs threshold\n(validation month)", xlabel="Threshold"); ax[0].legend()
ax[1].plot(sw["threshold"], sw["cost"] / len(y_val) * 10000, c="black"); ax[1].axvline(thr_cost, c="grey", ls=":");
ax[1].set(title=f"Assumed cost per 10,000 applications (missed fraud={COST_FN}, false alarm={COST_FP})", xlabel="
plt.tight_layout(); plt.show()

ops = {"Default 0.5": 0.5, "Max F1": thr_f1, "Lowest assumed cost": thr_cost, "5% false-alarm budget": thr_fpr5,
tt = pd.DataFrame({lbl: {**compute_metrics(y_test, s_test, t), "Threshold": t} for lbl, t in ops.items()}).T.astype(
tt["Alerts per 10k"] = (tt["TP"] + tt["FP"]) / len(y_test) * 10000; tt["Total cost"] = tt["FN"] * COST_FN + tt["F
display(tt[["Threshold", "Precision", "Recall", "F1", "TP", "FP", "FN", "Alerts per 10k", "Total cost"]].round(4)
rc_, d5 = tt.loc["Lowest assumed cost"], tt.loc["Default 0.5"]
interpret(f"Thresholds chosen on the validation month: max F1 {thr_f1:.3f}, lowest assumed cost {thr_cost:.3f}, 5%
f"On the test months it gives recall {rc_['Recall']:.3f} and precision {rc_['Precision']:.3f}, against
**Maximum fraud catch vs fewer false alarms:** a low threshold catches the most fraud but floods the al

```



	Threshold	Precision	Recall	F1	TP	FP	FN	Alerts per 10k	Total cost
Default 0.5	0.5000	0.2335	0.2503	0.2416	219.0000	719.0000	656.0000	152.3717	335,190.0000
Max F1	0.4671	0.2173	0.2731	0.2420	239.0000	861.0000	636.0000	178.6875	326,610.0000
Lowest assumed cost	0.0927	0.0729	0.7189	0.1324	629.0000	8,000.0000	246.0000	1,401.7219	203,000.0000
5% false-alarm budget	0.2078	0.1192	0.5154	0.1936	451.0000	3,333.0000	424.0000	614.6849	245,330.0000
1% false-alarm budget	0.4946	0.2308	0.2537	0.2417	222.0000	740.0000	653.0000	156.2703	333,900.0000

Interpretation

- Thresholds chosen on the validation month: max F1 0.467, lowest assumed cost 0.093, 5% false-alarm budget 0.208, 1% budget 0.495.
Recommended: 0.093 (lowest assumed cost).
- On the test months it gives recall 0.719 and precision 0.073, against recall 0.250 and precision 0.233 at 0.5, with about 1402 alerts per 10,000 applications.
- Maximum fraud catch vs fewer false alarms:** a low threshold catches the most fraud but floods the analysts; a high one keeps the queue short but lets fraud through. The best point depends on the real cost ratio and on how many alerts the team can review, which is why the costs at the top are editable.

Phase 7: Explainability and Business Interpretation

A fraud team must be able to explain **why** an application was flagged. For the best ML model we use two views: **built-in feature importance** and **permutation importance** (shuffle one column in the test data and measure how much PR-AUC drops, which works for any model). A third view, **SHAP**, shows the direction and size of each feature's effect; it is skipped safely if it fails in your environment.

```

est = RESULTS[best_ml_name]["estimator"]
native = pd.Series(est.feature_importances_ if hasattr(est, "feature_importances_") else np.abs(est.coef_.ravel())

```

```

native = (native / native.sum()).sort_values(ascending=False)
idx = np.random.RandomState(SEED).choice(len(y_test), size=min(40000, len(y_test)), replace=False)
perm = permutation_importance(est, X_test[idx], y_test[idx], scoring="average_precision", n_repeats=3, random_state=SEED)
perm_df = pd.Series(perm.importances_mean, index=FEAT_NAMES).sort_values(ascending=False)

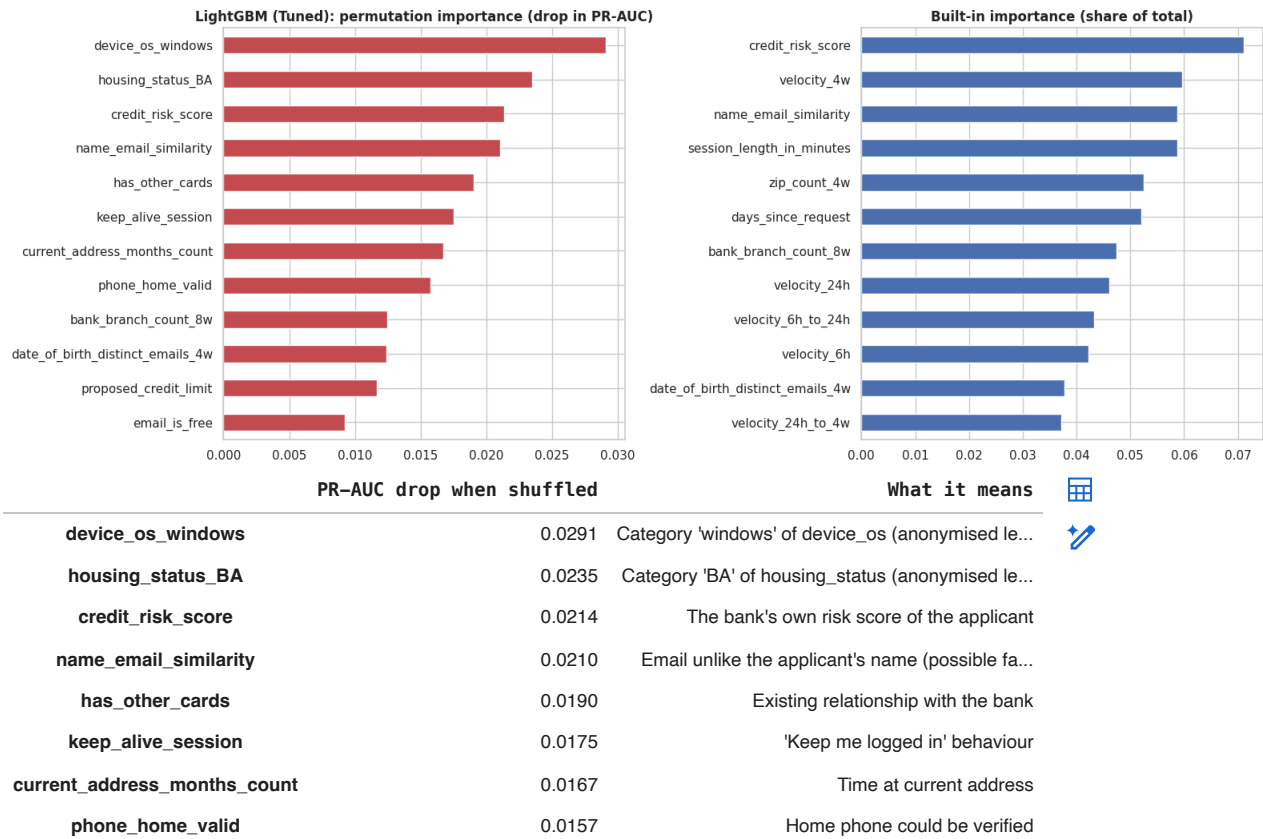
fig, ax = plt.subplots(1, 2, figsize=(16, 6))
perm_df.head(12)[::-1].plot(kind="barh", color=RED, ax=ax[0]); ax[0].set_title(f"{best_ml_name}: permutation importance")
native.head(12)[::-1].plot(kind="barh", color=BLUE, ax=ax[1]); ax[1].set_title("Built-in importance (share of total variance)")
plt.tight_layout(); plt.show()

MEANING = {"name_email_similarity": "Email unlike the applicant's name (possible fake identity)", "email_is_free_email": "Whether the email is a free email (Gmail, Yahoo, etc.)",
"velocity_6h": "Burst of applications in 6 hours (automated attack)", "velocity_24h": "Application bursts in the last 24 hours",
"velocity_6h_to_24h": "Acceleration of application bursts", "velocity_24h_to_4w": "Acceleration compared to the last 4 weeks",
"zip_count_4w": "Many applications from one area recently", "bank_branch_count_8w": "Unusual concentration of applications in a specific bank branch",
"date_of_birth_distinct_emails_4w": "Same birth date used with many emails (identity reuse)", "device_id": "Whether the device ID is unique",
"credit_risk_score": "The bank's own risk score of the applicant", "proposed_credit_limit": "High request for credit limit",
"credit_limit_to_income": "Limit that is high compared with income", "customer_age": "Age bucket (sense of age)",
"days_since_request": "Days since the application was made", "session_length_in_minutes": "Very short sessions",
"keep_alive_session": "'Keep me logged in' behaviour", "foreign_request": "Request from another country",
"phone_home_valid": "Home phone could be verified", "phone_mobile_valid": "Mobile phone could be verified",
"current_address_months_count": "Time at current address", "prev_address_months_count": "Time at previous address",
"intended_balance_amount": "Initial transfer amount", "n_missing_flags": "How many history fields are missing",
"address_to_age": "Time at address compared with age", "low_similarity_free_email": "Free email AND email similarity"}

def meaning(f):
    if f in MEANING: return MEANING[f]
    if f.endswith("_missing"): return f"History field '{f[:-8]}' is missing"
    for pre in CAT_RAW:
        if f.startswith(pre + "_"): return f"Category '{f[len(pre) + 1:]}' of {pre} (anonymised level)"
    return ""

top_perm_names, native_top = perm_df.head(5).index.tolist(), native.head(6).index.tolist()
tab = perm_df.head(8).to_frame("PR-AUC drop when shuffled"); tab["What it means"] = [meaning(f) for f in tab.index]
interpret(f"Most important columns by permutation: **{'', '.join(top_perm_names)}**. Shuffling `{perm_df.index[0]}`")
f"Top built-in features: {'', '.join(native_top)}.",
f"Engineered features in the permutation top 8: {[f for f in perm_df.head(8).index if f in NEW_FEATS] or ''}.
"Values near zero mean the model barely uses that column. Importance shows association, not cause, and")

```



Interpretation

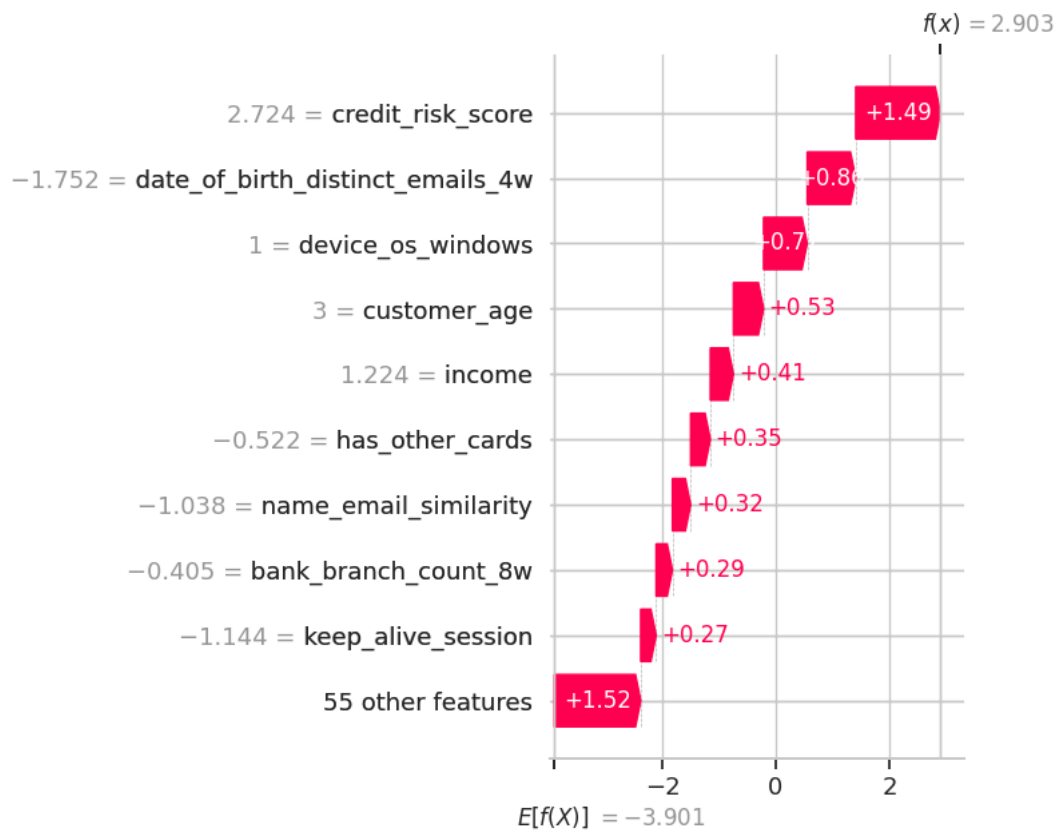
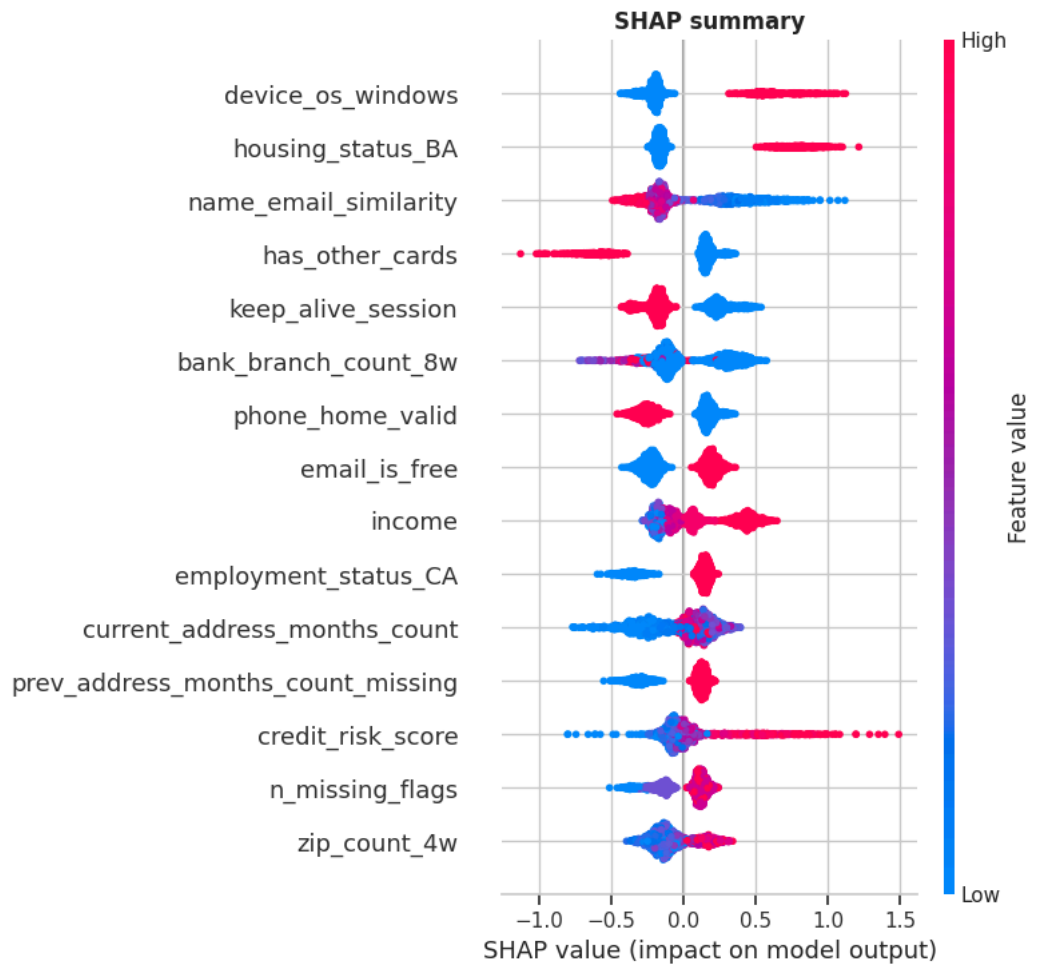
- Most important columns by permutation: **device_os_windows**, **housing_status_BA**, **credit_risk_score**, **name_email_similarity**, **has_other_cards**. Shuffling device_os_windows lowers PR-AUC by 0.0291, more than any other column.
- Top built-in features: credit_risk_score, velocity_4w, name_email_similarity, session_length_in_minutes, zip_count_4w, days_since_request.
- Engineered features in the permutation top 8: none.
- Values near zero mean the model barely uses that column. Importance shows association, not cause, and fraudsters change their behaviour over time.

Next steps:

[Generate code with tab](#)

[New interactive sheet](#)

```
# SHAP: global summary plus one local explanation (the highest-scoring truly fraudulent test application)
try:
    import shap
    fraud_idx = np.where(y_test == 1)[0]; top_local = int(fraud_idx[np.argmax(RESULTS[best_ml_name]["test"][fraud
    rows = sorted(set(np.random.RandomState(SEED).choice(len(X_test), min(1500, len(X_test))), replace=False).toli
    Xs = X_test[rows]
    expl = shap.TreeExplainer(est); sv = expl.shap_values(Xs)
    sv = sv[1] if isinstance(sv, list) else np.asarray(sv); sv = sv[:, :, 1] if sv.ndim == 3 else sv
    shap.summary_plot(sv, Xs, feature_names=FEAT_NAMES, max_display=15, show=False); plt.title("SHAP summary"); p
    i = rows.index(top_local); ev = np.ravel(expl.expected_value)[-1]
    shap.plots.waterfall(shap.Explanation(values=sv[i], base_values=ev, data=Xs[i], feature_names=list(FEAT_NAMES
    interpret("In the summary, each dot is one application; red = high feature value, blue = low; dots on the rig
    "The waterfall explains one confirmed fraud application: bars to the right raised its fraud score, l
except Exception as err:
    print("SHAP was skipped:", repr(err), "\nThe permutation importance above is already a valid explanation.")
```



Interpretation

- In the summary, each dot is one application; red = high feature value, blue = low; dots on the right push the prediction towards **fraud**.
- The waterfall explains one confirmed fraud application: bars to the right raised its fraud score, bars to the left lowered it. An analyst can read this as the reason for the flag.

7.1 What this means for the bank

What the model predicts: a fraud risk score for every new account application. **How a bank would use it:** applications with a low score are approved automatically; applications above the threshold go to an analyst queue **sorted from the highest score down**, and the very highest scores trigger an extra check (ID document, call-back) before the account is opened; confirmed fraud outcomes are saved and used to retrain the model. **False positives** delay a genuine customer and cost analyst time. **False negatives** let a fraudulent account open, which can mean unpaid credit, laundered money and fines, so they are usually the more expensive mistake. The model does **not replace** analysts: it focuses their limited time, and the explanations above tell them why an application looks risky.

```
final_prob = RESULTS[FINAL_NAME]["test"]; test_df = df_p.loc[te].reset_index(drop=True); n_test = len(y_test)
mt = compute_metrics(y_test, final_prob, RECOMMENDED_THR)
high_cut = max(thr_fpr1, RECOMMENDED_THR + 1e-6)
band = pd.cut(final_prob, [-0.01, RECOMMENDED_THR, high_cut, 1.01], labels=["Low (approve)", "Medium (analyst rev
bt = pd.DataFrame({"band": band, "actual": y_test}).groupby("band", observed=False)["actual"].agg(applications="s
bt["fraud_rate_%"] = (bt["actual_fraud"] / bt["applications"]).replace(0, np.nan) * 100).round(2); display(bt)

order = np.argsort(-final_prob)
view = test_df.loc[order[:10], ["customer_age", "income", "velocity_6h", "credit_risk_score", "name_email_similar
fraud_score=final_prob[order[:10]], actual=pd.Series(y_test[order[:10]]).map({0: "Genuine", 1: "Fraud"}).valu
print("TOP 10 HIGHEST-RISK TEST APPLICATIONS"); display(view)
cum = np.cumsum(y_test[order]) / max(y_test.sum(), 1)
gains = {k: float(cum[int(k * n_test) - 1]) for k in (0.01, 0.05, 0.10)}
interpret(f"At threshold {RECOMMENDED_THR:.3f} the model sends **{(mt['TP'] + mt['FP']) / n_test * 10000:.0f} of
f"An analyst checks about {(mt['TP'] + mt['FP']) / max(mt['TP'], 1):.1f} flagged applications to find o
f"Under the assumed costs, the total cost drops from {NO_MODEL_COST:,.0f} (no model) to {mt['FN'] * COS
"Scores are not exact percentages because class weights were used; use them for **ranking**.")
```

applications				actual_fraud	fraud_rate_%	
band						
Low (approve)				52931	246	0.4600
Medium (analyst review)				7667	407	5.3100
High (extra check first)				962	222	23.0800

TOP 10 HIGHEST-RISK TEST APPLICATIONS

	customer_age	income	velocity_6h	credit_risk_score	name_email_similarity	device_distinct_emails_8w	fraud
28595	70	0.9000	5,325.7568	320	0.2017		1.0000
55194	40	0.9000	2,048.1021	294	0.1287		1.0000
14195	60	0.9000	5,694.5890	229	0.3792		1.0000
26861	40	0.9000	5,076.3676	314	0.1408		1.0000
37590	30	0.9000	1,823.4605	302	0.7753		1.0000
44493	50	0.9000	1,767.0480	336	0.5061		1.0000
51443	40	0.9000	617.1379	281	0.0921		1.0000
5045	40	0.9000	1,799.9374	310	0.1091		2.0000
52111	60	0.8000	7,710.0989	322	0.0100		0.0000
58037	50	0.9000	1,295.8495	299	0.1042		1.0000

Interpretation

- At threshold 0.093 the model sends **1402 of every 10,000 applications** to review. It catches **629 of 875** frauds in the test months, misses **246** and